USED MEDIAS LLC
EMBEDDED SYSTEMS DIVISION

Flash Cache and Read-Only Data Placement

`UM-NATOS-011`

DOCUMENT	REVISION	STATUS	CLASSIFICATION
UM-NATOS-011	1.0 · 2026-08-14	**Complete, verified on hardware**	Internal

1. Abstract

Read-only data now lives in flash, mapped into the address space through the data cache, rather than occupying DRAM. This closes the prerequisite identified in UM-NATOS-010 §7.3 and deferred since UM-NATOS-004 §3.

The immediate saving is **1,248 bytes** — 0.7% of the heap, and not in itself a reason to do the work. The reason is what it unblocks: fonts, sprites and UI bitmaps can be built into the image at zero DRAM cost, and application bytecode can execute in place so an arena holds only an application's *data*. Doing this before M4 is considerably cheaper than retrofitting it once a bytecode format and a producer exist.

2. What was actually required

Less than expected, because the borrowed second-stage bootloader already does the hard part. It recognises load addresses in the flash-mapped ranges, programs the flash MMU to point at the image data in place, enables the cache, and only then jumps to the entry point. That is how every ESP-IDF application gets its read-only data mapped, and nat-os inherits it by producing an image of the same shape.

No cache configuration code was written. The change is entirely in the link map.

2.1 The DROM region

```
drom (r) : ORIGIN = 0x3F400020, LENGTH = 0x400000 - 0x20
```

The `0x20` offset is the part worth understanding, because getting it wrong produces an image that links, flashes, and then reads garbage.

The flash MMU maps in **64 KB pages**. The page that carries the read-only data begins at the start of the application image, and the first 32 bytes of that image are structural: a 24-byte image header followed by an 8-byte segment header. Starting the virtual region 32 bytes into the page makes the virtual address congruent with the flash offset, which is the relationship both `esptool` and the bootloader assume without checking.

Confirmed in the generated image:

```
Segment 1: len 0x004e0 load 0x3f400020 file_offs 0x00000018 [DROM]
Segment 2: len 0x00004 load 0x3ffb0000 file_offs 0x00000500 [BYTE_ACCESSIBLE,DRAM]
Segment 3: len 0x01510 load 0x40080000 file_offs 0x0000050c [IRAM]
```

`file_offs` reports each segment's *header*; the DROM payload therefore begins at `0x20`. Flashed at `0x10000`, the data sits at `0x10020` and maps to `0x3f400020`. The congruence holds.

2.2 What was deliberately not moved

Literal pools stay in IRAM. The build uses `-mtext-section-literals` precisely so they sit beside the code. `l32r` addresses backwards within 256 KB of the program counter, and a literal pool in flash would be far outside that window. This would most likely fail at link time — but "most likely" is not "certainly", and a literal silently resolving to the wrong address is a substantially worse outcome than a failed link.

`.data` and `.bss` are unaffected: both are writable, and flash is not.

2.3 Why the original placement no longer applies

`.rodata` was in DRAM because **IRAM cannot serve unaligned reads**, and string handling performs them constantly. That constraint is specific to IRAM. Flash mapped through the data cache has no such restriction, so the reasoning that forced the original placement simply does not carry over.

3. Verification

This change is close to self-verifying: every string the kernel prints is `.rodata`. If the mapping were wrong, the banner would be garbage or the board would fault before producing output. It is not, and it does not.

Check	Before	After
`.rodata` location	DRAM `0x3ffb0000`	flash, mapped `0x3f400020`
`.bss` start	`0x3ffb04f0`	`0x3ffb0010`
Heap usable	166,432 B	**167,680 B**
Image size	6,720 B	6,736 B

The heap grew by 1,248 bytes — **exactly** the size of the section that moved. An inexact match would have indicated something else shifting at the same time.

Image size grew by 16 bytes: one additional segment header, plus alignment. The `.rodata` bytes were always in the image; only their destination changed.

Regression. M3 self-test unchanged and passing in full — no leak across 10,000 cycles, arena bounds correct, exhaustion clean. M2 workload unchanged: 3,418 ticks, switches 1140/1139/1139, guards intact, `corrupt=0`.

4. Constraints this introduces

Mapping read-only data into flash is not free of consequence, and the consequences arrive later than the change:

- **Cache-off windows become hazardous.** Any future code that writes flash must disable the cache while doing so. During that window, *any* access to `.rodata` faults — including a string in an interrupt handler, and including the panic handler's own messages. ESP-IDF solves this with `IRAM_ATTR` and `DRAM_ATTR` placement for code and data that must survive a cache-off window. nat-os has no equivalent yet and does not need one until it writes flash, but the panic path is the case to fix first: a fault handler that cannot print because the cache is off is worse than no fault handler.
- **First-access latency.** A cache miss on flash is far slower than a DRAM read. For the tick handler this is jitter, currently unmeasured. It has not been an issue because the handler touches no `.rodata`, and that property is now worth preserving deliberately rather than by accident.
- **Cache configuration is now load-bearing.** The kernel depends on the bootloader having enabled the cache. Should L1 ever be replaced with our own code (UM-NATOS-001 §3), cache and MMU setup becomes our responsibility, and this report's assumptions become that replacement's requirements.

5. Metrics

Quantity	Value
`.rodata` relocated	1,248 B
DRAM reclaimed	1,248 B (0.7% of heap)
Heap after	167,680 B
Image growth	16 B
Flash used, total	6,736 B of 4 MB
Lines of C written	0
Lines of linker script changed	~20

6. What this does not establish

- **No asset pipeline.** Nothing yet converts a font or bitmap into a linkable read-only object. The mechanism exists; the tooling does not.
- **No execute-in-place for code.** Only data is mapped. An IROM segment for kernel code has not been attempted, and would interact with the `l32r` constraint of §2.2.
- **No flash writing.** Nothing writes flash at runtime, so the cache-off hazard of §4 is latent rather than live.
- **No latency measurement.** The cache-miss cost has not been quantified.

7. References

- UM-NATOS-004 §3 — cache configuration, deferred there and closed here
- UM-NATOS-010 §7.3 — the analysis that made this a prerequisite for M4
- UM-NATOS-001 §3 — the borrowed-L1 boundary this change leans on
- `kernel/linker.ld` — `drom` region and `.flash.rodata`